

Part 2: System Design — Notification Service

Scalable notification architecture for email, SMS, and push notifications

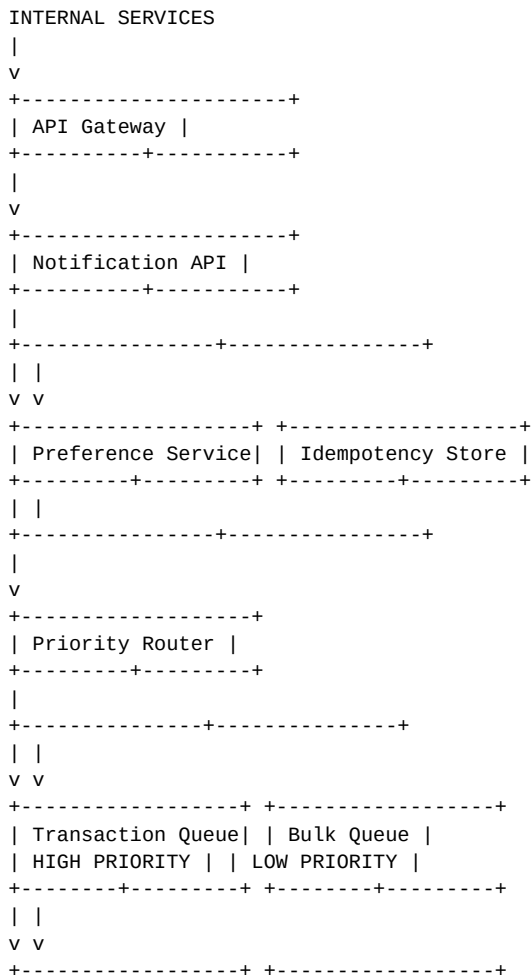
1. System Overview

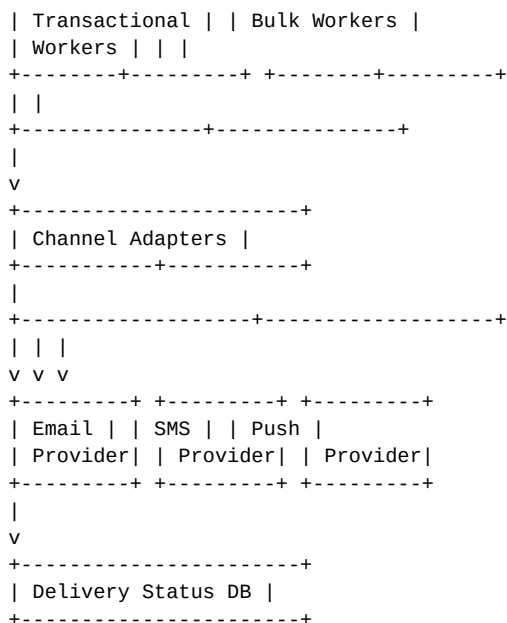
The notification service is a shared internal platform used by product services to deliver notifications to millions of users through email, SMS, and push. The design prioritises reliability, scalability, extensibility, and user control. Notifications are processed asynchronously through durable queues so traffic spikes can be absorbed without overwhelming application servers or third-party providers.

2. Requirements

- Support email, SMS, and push notifications with an extensible channel architecture.
- Respect user opt-outs, per-channel preferences, and quiet hours.
- Prevent duplicate notifications through idempotency.
- Track sent, delivered, and failed states.
- Process approximately 10 million notifications per day and absorb large spikes.
- Provide at-least-once delivery without silently losing queued notifications.
- Keep transactional notifications near real-time while allowing bulk traffic to be delayed.
- Handle unreliable, rate-limited, or unavailable third-party providers.

3. High-Level Architecture





4. Major Components

Component	Responsibility
API Gateway	Authenticates internal callers, applies basic traffic controls, and distributes requests.
Notification API	Validates requests, creates notification jobs, applies priority, and coordinates processing.
Preference Service	Checks opt-outs, channel preferences, and quiet-hour rules.
Idempotency Store	Records unique notification/event IDs so duplicate requests do not create duplicate sends.
Priority Router	Routes urgent transactional traffic separately from bulk traffic.
Transactional Queue	Durably stores high-priority notifications such as OTPs and password resets.
Bulk Queue	Absorbs large marketing and campaign workloads that can tolerate delay.
Workers	Consume jobs, apply delivery rules, call providers, and update delivery state.
Channel Adapters	Provide a common interface for email, SMS, and push while hiding provider-specific APIs.
Delivery Status DB	Stores notification lifecycle states, provider responses, timestamps, and audit information.

5. Notification Flow

- An internal service sends a notification request to the Notification API.
- The API validates the request and checks the user's preferences and quiet hours.
- An idempotency key is checked to determine whether the same notification has already been accepted or processed.
- The request is assigned a priority and placed on the transactional or bulk queue.
- A worker consumes the job and selects the appropriate channel adapter.
- The adapter calls the external provider and records the provider response.
- The notification status is updated as sent, delivered, or failed.
- Transient failures are retried; exhausted or permanent failures are moved to a dead-letter queue.

6. Scale: 10 Million Notifications per Day

Ten million notifications per day averages approximately 116 notifications per second. The average rate is not the main challenge because marketing campaigns can create much larger bursts. Durable queues decouple incoming traffic from provider throughput. Workers can be scaled horizontally according to queue depth, while provider-specific limits are respected.

7. Transactional vs Bulk Traffic

Transactional notifications such as password resets, OTPs, security alerts, and important account events are placed in a high-priority queue. Dedicated or reserved worker capacity keeps these messages near real-time. Bulk campaigns use a separate queue and worker pool, allowing them to build up during spikes without starving transactional traffic.

8. At-Least-Once Delivery

A notification is written to durable queue storage before processing. A worker acknowledges or removes the message only after processing reaches a safe state. If the worker crashes before acknowledgement, the queue can make the message available again. This provides at-least-once processing, meaning a message may be processed more than once but should not be silently lost.

9. De-duplication and Idempotency

Every notification request receives a unique idempotency key or event ID. The system stores this key with the notification record. Repeated requests with the same key return the existing result rather than creating another notification. Worker operations should also be idempotent because queue retries can cause the same job to be delivered more than once.

10. Retry Strategy

Temporary provider errors, timeouts, and rate-limit responses should be retried using exponential backoff with jitter. A maximum retry count prevents infinite attempts. A typical strategy could be:

Attempt	Approximate delay
1	Immediate
2	1 second
3	5 seconds
4	30 seconds
5+	Dead-letter queue / manual recovery

Permanent failures, such as invalid recipient data, should not be retried indefinitely. Failed messages should retain the failure reason for investigation.

11. Provider Failover

Third-party providers may be unavailable or rate-limited. The service should monitor provider health and response codes. If a provider experiences a temporary outage, messages can be retried later or routed to a secondary provider where contracts and channel capabilities allow. Circuit breakers can temporarily stop sending to an unhealthy provider and prevent cascading failures.

12. User Preferences and Quiet Hours

Before a notification enters a delivery queue, the service checks whether the user has opted out of the channel and whether that notification type is permitted. Quiet-hour rules can delay non-critical notifications until the allowed delivery period. Critical transactional notifications can follow a separate policy when business requirements permit them to bypass normal quiet hours.

13. Delivery Status Tracking

The notification record should maintain a lifecycle such as accepted → queued → sending → sent → delivered, or failed. Provider callbacks/webhooks can update delivered status where supported. Provider message IDs, timestamps, error codes, and retry counts should be retained for observability and auditing.

14. Extensible Channel Architecture

The core service should use a common adapter interface so provider-specific logic is isolated from business logic. For example:

```
NotificationService
  ■■■ EmailAdapter
  ■■■ SMSAdapter
  ■■■ PushAdapter
  ■■■ Future: WhatsAppAdapter
```

Adding another channel therefore requires implementing a new adapter rather than changing the queueing, preference, idempotency, and routing components.

15. Data Stores

Store	Purpose	Reason
Relational database	Notification records, preferences, idempotency keys, and users	Strong consistency and queryability for critical metadata.
Durable message queues	Pending notification jobs	Absorbs spikes and supports retry/redelivery.
Distributed cache	Optional rate limits, short-lived state, caching	Fast access across multiple application instances.

16. Failure Scenarios

Failure	Response
Worker crashes	Unacknowledged queue messages are redelivered to another worker.
Provider timeout	Retry with exponential backoff and jitter.
Provider rate limit	Respect retry-after information where available and throttle the provider-specific worker pool.
Provider outage	Use circuit breaking and fail over to another provider where possible.
Database outage	Use durable infrastructure, replicas/backups, and retry transient database operations.
Marketing spike	Bulk queue absorbs backlog while reserved transactional capacity remains available.
Duplicate request	Idempotency key prevents creation of another notification.

17. Key Tradeoffs

- Separate transactional and bulk queues improve reliability and latency isolation but add infrastructure and operational complexity.
- Asynchronous processing improves API response time and spike handling but means delivery is eventually consistent.
- At-least-once delivery increases reliability but requires idempotency to prevent duplicates.
- Multiple providers improve availability but increase integration, cost, and monitoring requirements.
- A relational database provides strong consistency for metadata but requires scaling and backup planning at high volume.
- Channel adapters improve extensibility but introduce an additional abstraction layer.

18. Future Improvements

Future improvements could include automatic worker autoscaling based on queue depth, multi-region deployment, provider-specific circuit breakers, distributed tracing, advanced delivery analytics, stronger disaster recovery, scheduled notifications, template management, and an operational dashboard. The service could also support additional channels such as WhatsApp while keeping the core architecture unchanged.

19. Conclusion

The proposed notification service uses durable queues, separate priority paths, idempotency, retries, provider failover, and channel adapters to meet the requirements of a large-scale notification platform. The architecture can process millions of notifications while protecting near-real-time transactional traffic from bulk workloads. Most importantly, the design accepts that third-party providers and workers can fail and builds recovery into the normal processing model.